Prezentacja projektu

Co pokazać w przeglądarce i gdzie znaleźć kod

Przewodnik po wszystkich technicznych punktach zaliczenia oraz dodatkach sklepu ByteGear. Adresy stron i ścieżki do źródeł są klikalne. Link do kodu prowadzi do GitHub; na komputerze otwórz folder bytegear w Visual Studio Code.

Jak szybko znaleźć plik

Ctrl+P - wklej ścieżkę z tego dokumentu i Enter. **Ctrl+F** - wyszukaj nazwę metody lub fragment podany jako „Znajdź". Ścieżki są względem folderu projektu. Metody są pewniejsze niż numery linii, które zmieniają się po aktualizacji.

Przed pokazem

Uruchom MariaDB, sklep na porcie 8002 i Mailpit na 8025 zgodnie z pierwszym PDF-em. Przygotuj VS Code i phpMyAdmin. Konta admin@example.test, moderator@example.test i customer@example.test mają hasło **DemoSklep123!**, o ile nie zostało zmienione.

Do CRUD utwórz osobne konto i produkt testowy; nie usuwaj kont demo. Używaj oddzielnych profili przeglądarki dla klienta i administratora albo wylogowuj się między rolami. Nowe konto aktywuj przez Mailpit.

Wymaganie prowadzącego	Gdzie w przewodniku
HTML5, CSS3, PHP 8.x, MariaDB, Laravel	s. 2
Rejestracja, logowanie, zapis i hashowanie haseł	s. 3
Co najmniej 2 role, różne strony po logowaniu	s. 4 (są 3 role)
CRUD użytkowników, lista i pojedynczy użytkownik	s. 5
Walidacja wymaganych pól	s. 3, 5 i 8
Koszyk w sesji, produkty, zamawianie	s. 6 i 7
Aktywacja e-mailem, regex, SQL Injection	s. 4 i 8
Dodatki: profil, reset, zdjęcia, API i testy	s. 6, 9 i 10

Samodzielność, oddanie w terminie i obrona są obowiązkami studenta; aplikacja ich nie potwierdza. Przećwicz wyjaśnianie kodu. Według Twojej informacji prowadzący nie wymaga modyfikacji kodu bez Internetu ze starego opisu kursu.

2. Technologia, baza i MVC

Pokaż katalog sklepu oraz terminal

```
php --version
php artisan --version
php artisan migrate:status
php artisan route:list
```

Wynik: PHP 8.x, Laravel Framework 12.x, wykonane migracje i lista tras. W phpMyAdmin otwórz bazę bytegear: users, categories, products, orders, order_items oraz sessions.

composer.json

Znajdź: **php, ext-gd, laravel/framework**

config/database.php

Znajdź: **mysql oraz env('DB_DATABASE')**

database/migrations/0001_01_01_000000_create_users_table.php

Znajdź: **users, sessions, password_reset_tokens**

database/migrations/2026_10_06_000001_create_shop_tables.php

Znajdź: **role, products, orders, order_items**

database/seeders/ShopSeeder.php

Znajdź: **run()**

Wyjaśnij: .env ma lokalne DB_HOST, DB_DATABASE, DB_USERNAME i DB_PASSWORD. Driver mysql obsługuje MariaDB. Migracje tworzą strukturę, seeder dodaje dane pokazowe. Na publicznym ekranie pokazuj .env.example zamiast sekretów swojego .env.

Jeden przykład przepływu MVC

Otwórz http://127.0.0.1:8002/. Wpisz nazwę produktu w wyszukiwarce.

routes/web.php

Znajdź: **CatalogController::class, 'index'**

app/Http/Controllers/CatalogController.php

Znajdź: **index() i view('catalog.index'**

app/Models/Product.php

Znajdź: **category()**

resources/views/catalog/index.blade.php

Znajdź: **produkty i formularz filtrów**

resources/views/layouts/app.blade.php

Znajdź: **DOCTYPE, asset('css/shop.css'), @yield**

public/css/shop.css

Znajdź: **style i @media**

Powiedz: trasa kieruje żądanie do kontrolera, kontroler odczytuje modele z bazy i przekazuje dane do Blade, który generuje HTML5. CSS3 nadaje wygląd. Obrazy i style są lokalne; nie trzeba npm.

3. Rejestracja i logowanie

A. Nowe konto i zapis użytkownika

Przeglądarka: http://127.0.0.1:8002/rejestracja

Zrób: Jako gość utwórz konto, np. pokaz@example.test, z hasłem PokazSklepu123! i potwierdzeniem. Za każdym kolejnym pokazem użyj nowego adresu.

Wynik: Powstaje rekord users z rolą customer i niepotwierdzonym e-mailem. Wiadomość czeka w Mailpit. Nieaktywne konto nie może jeszcze zamawiać.

resources/views/auth/register.blade.php

Znajdź: **POST i @csrf**

app/Http/Controllers/AuthController.php

Znajdź: **register(), validate(), role = customer, User::create**

app/Models/User.php

Znajdź: **casts() i password => hashed**

Powiedz: Hasło jest hashowane jednokierunkowo, a nie zapisywane jawnie. Cast modelu działa przy rejestracji i tworzeniu konta w panelu. Rola jest ustalana na serwerze; formularz rejestracji nie może nadać admina.

B. Logowanie i wylogowanie

Przeglądarka: http://127.0.0.1:8002/logowanie

Zrób: Spróbuj błędnego hasła, potem poprawnego. Aktywuj nowe konto według s. 4 albo użyj konta demo. Na końcu kliknij Wyloguj.

Wynik: Błędne dane nie logują. Poprawne dane tworzą zalogowaną sesję, a przycisk Wyloguj ją unieważnia.

app/Http/Controllers/AuthController.php

Znajdź: **login(): Auth::attempt(), regenerate(); logout(): invalidate()**

config/auth.php

Znajdź: **guards.web, providers.users**

Powiedz: Auth::attempt sprawdza hasło względem hasha. Po logowaniu zmieniamy identyfikator sesji. Wylogowanie jest żądaniem POST z tokenem CSRF.

Pokaż walidację już na tym etapie

W formularzu rejestracji wpisz niepoprawny e-mail, różne potwierdzenie lub hasło abc. Pokaż komunikaty. W kodzie wskaż required, email, unique, min, max, confirmed oraz regex. Walidacja serwera działa także wtedy, gdy ktoś ominie HTML required.

W phpMyAdmin można pokazać, że users.password zawiera hash, a nie hasło PokazSklepu123!. Nie wyświetlaj prywatnych kont ani eksportu danych na publicznym nagraniu.

4. Aktywacja i poziomy uprawnień

A. Potwierdzenie adresu e-mail

Przeglądarka: http://127.0.0.1:8002/konto

Zrób: Zaloguj się na nowo zarejestrowane konto. Otwórz Mailpit, znajdź jego wiadomość i kliknij Aktywuj konto. Jeśli trzeba się zalogować, użyj tego samego konta.

Wynik: Znika ekran oczekiwania, email_verified_at otrzymuje datę. Link ma podpis i ważność 60 minut. Można wysłać go ponownie.

app/Models/User.php

Znajdź: **implements MustVerifyEmail**

app/Services/AccountMail.php

Znajdź: **verify()**

app/Http/Controllers/VerificationController.php

Znajdź: **notice(), verify(), send(), fulfill()**

routes/web.php

Znajdź: **signed, verified, verification.verify**

app/Providers/AppServiceProvider.php

Znajdź: **VerifyEmail::toMailUsing**

Powiedz: To rzeczywisty mechanizm aktywacji przez wiadomość e-mail, z lokalnym odbiornikiem SMTP. Mailpit nie dostarcza wiadomości do Internetu. Gdy prowadzący wymaga zewnętrznej skrzynki, ustaw prawdziwy SMTP w .env; mechanizm aktywacji pozostaje ten sam.

B. Trzy role i różne ekrany

Przeglądarka: http://127.0.0.1:8002/konto

Zrób: Kolejno zaloguj customer@example.test, moderator@example.test i admin@example.test. Za każdym razem pokaż stronę po logowaniu. Jako klient wpisz ręcznie /panel/users i /panel/products.

Wynik: Klient widzi konto i zamówienia, moderator panel produktów, administrator panel użytkowników. Próba wejścia klienta do panelu pracownika kończy się 403.

app/Http/Controllers/OrderController.php

Znajdź: **dashboard() i match**

app/Http/Middleware/RequireRole.php

Znajdź: **handle(), abort_unless(), 403**

bootstrap/app.php

Znajdź: **alias role**

routes/web.php

Znajdź: **role:admin,moderator oraz role:admin**

Powiedz: Uprawnienia są sprawdzane na serwerze. Ukrycie linku w menu nie wystarcza. Trzy role spełniają wymaganie co najmniej dwóch poziomów.

5. CRUD użytkowników w panelu

A. Lista, szczegóły, dodawanie i edycja

Przeglądarka: http://127.0.0.1:8002/panel/users

Zrób: Zaloguj administratora. Pokaż listę i przycisk szczegółów wybranego konta. Dodaj konto crud-pokaz@example.test, następnie zmień jego imię i rolę. Użyj własnego nieużywanego adresu, jeśli taki już istnieje.

Wynik: Lista zawiera nowe konto; szczegóły pokazują pojedynczego użytkownika. Edycja zapisuje zmianę. Nowe konto otrzymuje hash hasła i wiadomość aktywacyjną.

app/Http/Controllers/AdminUserController.php

Znajdź: **index(), show(), create()/store(), edit()/update(), data()**

resources/views/admin/users/index.blade.php

Znajdź: **lista i działania**

resources/views/admin/users/show.blade.php

Znajdź: **pojedynczy użytkownik**

resources/views/admin/users/form.blade.php

Znajdź: **pola formularza**

routes/web.php

Znajdź: **resource('users'**

Powiedz: CRUD to Create, Read, Update, Delete. Formularze i zapis są osobnymi akcjami. Tę samą ochronę hasła zapewnia model User. Zmiana adresu e-mail wymaga ponownej aktywacji.

B. Usunięcie i kontrola błędów

Przeglądarka: http://127.0.0.1:8002/panel/users

Zrób: Usuń tylko nowo utworzone konto testowe i pokaż jego brak na liście. Spróbuj dodać konto z istniejącym e-mailem albo niepoprawnymi danymi.

Wynik: Duplikat i błędne dane są odrzucane. Administrator nie może usunąć siebie ani pozbawić aplikacji ostatniego administratora. Historyczne zamówienia nie znikają po usunięciu konta.

app/Http/Controllers/AdminUserController.php

Znajdź: **destroy(), data(), DB::transaction**

database/migrations/2026_10_06_000001_create_shop_tables.php

Znajdź: **nullOnDelete() dla user_id**

lang/pl/validation.php

Znajdź: **polskie komunikaty**

resources/views/layouts/app.blade.php

Znajdź: **$errors->any()**

Powiedz: Kontrola danych i uprawnień działa w PHP. Walidacja zapobiega niepoprawnemu zapisowi, a klucze obce chronią relacje. Historia zakupu jest niezależna od dalszego istnienia konta.

6. Produkty, zdjęcia i koszyk

A. Dodanie, edycja i wycofanie produktu

Przeglądarka: http://127.0.0.1:8002/panel/products

Zrób: Jako moderator lub admin dodaj BG Pokaz USB (SKU POKAZ-USB-1, cena 49.90, stan 5, dowolna kategoria). Opcjonalnie wgraj własne JPG/PNG/WebP. Pokaż go w katalogu, edytuj cenę, potem wycofaj i przywróć aktywność.

Wynik: Produkt jest zapisany i widoczny w sklepie. Po wycofaniu znika z oferty. Pozostaje w panelu i bazie, a stare zamówienia nie tracą pozycji.

app/Http/Controllers/AdminProductController.php

Znajdź: **store(), update(), destroy(), data()**

app/Services/ProductImage.php

Znajdź: **store()**

app/Models/Product.php

Znajdź: **imagePath()**

app/Http/Controllers/CatalogController.php

Znajdź: **where('active', true), image()**

Powiedz: Usuwanie ze sklepu oznacza wycofanie active=false, z zachowaniem historii. Zdjęcia są walidowane do 4 MB i przetwarzane przez GD. Własne pliki są w storage/app/private/product-images, obrazy demo w public/images/products.

B. Koszyk przechowywany w sesji

Przeglądarka: http://127.0.0.1:8002/koszyk

Zrób: W katalogu dodaj dostępny produkt. Zmień liczbę sztuk, odśwież stronę i usuń pozycję. Ponownie dodaj produkt, aby przygotować zakup. Koszyk może działać również bez logowania.

Wynik: Po odświeżeniu pozycje zostają w bieżącej sesji. Ilość i suma się zmieniają. Za duża ilość lub brak stanu są odrzucane.

app/Http/Controllers/CartController.php

Znajdź: **add(), update(), destroy(), session()->put('cart'**

app/Services/Cart.php

Znajdź: **rows(), total()**

config/session.php

Znajdź: **driver**

resources/views/cart.blade.php

Znajdź: **formularze ilości i usuwania**

Powiedz: W sesji trzymamy ID i ilości. Ceny są odczytywane z bazy; klient nie może wysłać własnej ceny. SESSION_DRIVER=database oznacza zapis sesji w MariaDB, ale nadal jest to koszyk w sesji.

7. Zamawianie i historia zakupów

A. Klient składa zamówienie

Przeglądarka: http://127.0.0.1:8002/zamowienie

Zrób: Zaloguj aktywnego klienta, dodaj produkt do koszyka i przejdź do zamówienia. Wpisz dane testowe, np. Test Pokazu, Pokazowa 1, 00-001 Warszawa. Najpierw wpisz błędny kod, potem popraw i zatwierdź zakup.

Wynik: Błędny kod pocztowy daje komunikat. Poprawny zakup zapisuje zamówienie i pozycje, zmniejsza stan produktu i czyści koszyk. Potwierdzenie trafia do Mailpit.

app/Http/Controllers/OrderController.php

Znajdź: **checkout(), store(), validate(), DB::transaction(), lockForUpdate()**

resources/views/checkout.blade.php

Znajdź: **pola i checkout_token**

app/Services/OrderMail.php

Znajdź: **send()**

resources/views/mail/order-confirmation.blade.php

Znajdź: **treść potwierdzenia**

Powiedz: Ceny w bazie są całkowitymi groszami. Transakcja zapisuje całość lub wycofuje zmiany; blokady chronią magazyn. Formularz ma ochronę przed powtórnym zapisem. Awaria SMTP nie cofa zakupu; nieudaną wysyłkę można ponowić.

B. Lista, status i anulowanie

Przeglądarka: http://127.0.0.1:8002/zamowienia

Zrób: Otwórz szczegóły własnego zamówienia. Dla jednego nowego zamówienia potwierdź anulowanie. Dla drugiego pokaż w /panel/orders zmianę na realizowane, potem zrealizowane; wróć do widoku klienta.

Wynik: Historia pozostaje. Anulowanie nowego zamówienia zwraca zapas tylko raz. Zamówienia w realizacji klient już nie anuluje. Pracownik może zmieniać status do następnego etapu.

app/Http/Controllers/OrderController.php

Znajdź: **index(), show(), cancel(), increment('stock'**

app/Http/Controllers/AdminOrderController.php

Znajdź: **update()**

app/Models/Order.php

Znajdź: **STATUSES, items(), user()**

resources/views/orders/show.blade.php

Znajdź: **szczegóły i przyciski**

Powiedz: order_items przechowuje historyczne nazwy i ceny. Zmiana lub wycofanie produktu nie zmienia starego zamówienia. show() i cancel() kontrolują właściciela; klient nie ogląda cudzych zakupów.

8. Walidacja, regex i bezpieczeństwo

A. Wyrażenia regularne - wymaganie na wyższą ocenę

Pokaż niepoprawne hasło w rejestracji, kod pocztowy w zamówieniu oraz małe litery/spację w SKU produktu. Reguły PHP odrzucają dane; polskie błędy są widoczne na stronie.

app/Http/Controllers/AuthController.php

Znajdź: **register(): regex hasła**

app/Http/Controllers/OrderController.php

Znajdź: **store(): regex kodu pocztowego**

app/Http/Controllers/AdminProductController.php

Znajdź: **data(): regex SKU**

```
Hasło: /^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9]).+$/s
Kod pocztowy: /^[0-9]{2}-[0-9]{3}$/
SKU: /^[A-Z0-9-]+$/
```

Hasło wymaga małej i wielkiej litery oraz cyfry; długość sprawdzają min/max. Kod ma format 00-000; SKU zawiera wielkie litery, cyfry i myślniki.

B. SQL Injection - wymaganie na wyższą ocenę

Na stronie katalogu wpisz w wyszukiwarce tekst **' OR 1=1 --**. Nie uzyskujesz dostępu do bazy; to tylko szukana wartość. Kod i test są lepszym dowodem niż sam widok pustej listy.

app/Http/Controllers/CatalogController.php

Znajdź: **index(): where('name', 'like'...), validate(sort), match**

tests/Feature/ShopTest.php

Znajdź: **test_html_is_escaped_and_search_injection_does_not_change_database**

Powiedz: Eloquent wiąże wartości jako parametry SQL. Znaki % są częścią wartości wyszukiwania, a nie doklejonym kodem SQL. Sortowanie wybiera kolumnę z zamkniętej listy. Nie budujemy zapytań SQL przez sklejanie danych użytkownika.

C. CSRF, XSS i kontrola dostępu

resources/views/auth/register.blade.php

Znajdź: **@csrf**

resources/views/catalog/show.blade.php

Znajdź: **{{ ... }}**

routes/web.php

Znajdź: **auth, auth.session, verified, role, throttle**

CSRF chroni akcje formularzy przed obcymi żądaniami. Blade {{ ... }} ucieka HTML, ograniczając XSS. Middleware sprawdza login, aktywację i rolę przed kontrolerem. To osobne mechanizmy, a nie trzy nazwy ochrony przed SQL Injection.

9. Dodatki do zaprezentowania

A. Profil i zmiana hasła

Przeglądarka: http://127.0.0.1:8002/profil

Zrób: Jako klient zmień imię. Opcjonalnie zmień e-mail, wpisując bieżące hasło, i pokaż nową wiadomość aktywacyjną. Zmień hasło na inną poprawną wartość; zanotuj je, jeśli modyfikujesz konto demo.

Wynik: Imię zapisuje się. Zmiana e-maila wymaga ponownego potwierdzenia. Zmiana hasła sprawdza stare hasło i usuwa pozostałe sesje, zachowując bieżącą.

app/Http/Controllers/ProfileController.php

Znajdź: **edit(), update(), password()**

resources/views/auth/profile.blade.php

Znajdź: **formularze profilu i hasła**

Powiedz: Użytkownik edytuje tylko dozwolone pola; nie może nadać sobie roli. Nowe hasło przechodzi walidację i hashowanie.

B. Odzyskanie zapomnianego hasła

Przeglądarka: http://127.0.0.1:8002/zapomniane-haslo

Zrób: Wyloguj się. Podaj adres konta testowego, otwórz wiadomość w Mailpit i ustaw nowe hasło. Zaloguj się nowym hasłem.

Wynik: Token działa jednorazowo przez 60 minut. Poprzednie sesje są unieważniane. Odpowiedź formularza nie ujawnia, czy dany e-mail istnieje.

app/Http/Controllers/PasswordController.php

Znajdź: **email(), update(), Password::sendResetLink(), Password::reset()**

config/auth.php

Znajdź: **passwords.users: expire, throttle**

Powiedz: Reset korzysta z brokera Laravel i tabeli password_reset_tokens. To inny mechanizm niż aktywacja konta.

C. Wygląd, wyszukiwanie i konfiguracja marki

W katalogu pokaż kategorię, sortowanie, filtr dostępności, ceny i paginację. Zwęź okno lub użyj widoku urządzenia w przeglądarce. Sprawdź nawigację Tab. Style są w public/css/shop.css, layout w resources/views/layouts/app.blade.php.

Nazwa sklepu pochodzi z APP_NAME w .env przez config/app.php i config('app.name'). Po zmianie wykonaj php artisan config:clear. Nie trzeba zmieniać nazwy bazy ani repozytorium. Modele BG i wizualizacje są demonstracyjne; sklep nie przetwarza rzeczywistych płatności.

10. API, testy i krótki plan obrony

API NBP - dodatek pokazujący materiał z kursu

Na szczegółach dowolnego produktu kliknij cenę w EUR. Przykład dla początkowych danych: http://127.0.0.1:8002/produkty/1/euro. Jeśli produkt o ID 1 nie istnieje lub jest wycofany, wybierz inny z katalogu.

app/Http/Controllers/CatalogController.php

Znajdź: **euro(), timeout(8)**

resources/views/catalog/euro.blade.php

Znajdź: **kurs EUR i komunikat o błędzie**

Kurs pochodzi z API NBP. Brak Internetu wyświetla komunikat; nie blokuje zamawiania w PLN. API i automatyczne testy są dodatkami, a nie osobnymi minimalnymi wymaganiami sklepu.

Testy automatyczne

```
php artisan test
```

Ostatni sprawdzony wynik tej wersji: **41 testów, 310 asercji**. Liczby mogą zmienić się po dalszych zmianach. Testy używają SQLite w pamięci, oddzielnie od MariaDB sklepu.

phpunit.xml

Znajdź: **DB_CONNECTION=sqlite, DB_DATABASE=:memory:**

tests/Feature/ShopTest.php

Znajdź: **CRUD, role, koszyk, zamówienia, błędne dane**

tests/Feature/AccountSecurityTest.php

Znajdź: **aktywacja, podpis, reset, sesje**

tests/Feature/ShopExtensionsTest.php

Znajdź: **profil, zdjęcia, anulowanie i wiadomości**

Prezentacja minimum w około 10-15 minut

1. Katalog i MVC; wersje PHP/Laravel oraz tabele.
2. Rejestracja z błędnymi danymi, poprawny zapis, hash i Mailpit.
3. Logowanie trzech ról i odmowa dostępu klientowi.
4. Admin: lista, szczegóły, dodanie, edycja i usunięcie konta testowego.
5. Moderator: dodanie i wycofanie produktu.
6. Klient: koszyk, zakup, historia; regex i zapytanie parametryzowane.
7. Jeśli jest czas: profil, anulowanie, wiadomość, API i testy.

Co świadomie wyjaśnić prowadzącemu

Produkty wycofujemy z oferty zamiast kasować historię. Poczta podczas pokazu trafia do lokalnego Mailpit; zewnętrzny SMTP jest opcją konfiguracji. Każdą funkcję umiej wskazać jako: trasa - kontroler - model/baza - widok. Ocenę i zaliczenie przyznaje prowadzący po prezentacji, nie sam wynik testów.

Repozytorium: https://github.com/pzx119592/WSB_Projekt_ByteGear. Dokumenty PDF i instrukcje TXT można pobrać razem z kodem.